

Computer Graphics Project #2

Diana Livengood

Dillon Fiske

Matthew Thomas

10/2/2026

- Diana Livengood

- Terrain Generation)

- The terrain is created via a planar geometry geometry type, whose width and height are each equal to 100, giving us a square. Instead of being composed of the 4 vertices normally found in a flat cube, the geometry is instead composed of 129 vertices along each axis (128 vertices + 1 edge vertex to compose the shape), giving a total end result of 16,641 vertices.
- Next the vertex shader distorts the position of the Z vertice, warping the square into the three dimensional space. It does this by using a randomized seed created by `Math.random()*1000`. First the X and Y positions of the geometry are melded together into a singular number. The X value is weighted at $Y * 10$, whereas the Y value is weighted at $Y * 80$. If the X and Y values are weighted equally then a pattern emerges, creating a boring geometry that does not feel randomized. The value is multiplied by the random seed to generate pseudorandomness, and by a value of 40,000 to increase said randomness. This value is then used to update the Z coordinate of each vertex, randomizing their heights and giving us the random terrain.

After the terrain is randomized it has a `sin()` function continuously applied to it to generate the wave effect. This wave is controlled by a `uTime` variable which is updated in the animation loop.

- The terrain is then rotated 90 degrees so that it occupies the XZ space instead of the XY space. This makes the previous Z vertex randomization occupy height instead of "width".
- The fragment shader colors the terrain blue. It also handles the highlighting function, which will be covered in Matthew's section.
- The terrain is then added to the scene.

- Camera Movement)

- The WASD camera movement is handled by event listeners that update whether or not the key is being pressed. When the key is pressed the animation loop moves the camera in its corresponding direction X amount within delta time. This movement is not based on changing the camera's X/Y/Z value, but is handled by the `PointerLockControls` import.

This import allows camera movement corresponding to the

direction it is facing. Control is used to make the camera descend and spacebar is used to make the camera ascend.

- The camera's facing direction movement is also handled by this import automatically.
- The controls lock when switching camera view to prevent disconnects for the user while swapping cameras.

- **Matthew Thomas**

- **Github)**

- I created and managed the github repository, including using the Pages feature to make an easy link to our HTML file on a live website so that it could be accessed with downloading it locally

- **Terrain Highlighting and Scaling)**

- The program searches for the point on the plane mesh that lines up with the satellite, then changes the color and applies a scalar to the area around that point. It also checks whether the satellite is above or below the plane, and inverts the direction of the scaling appropriately so that the mesh always scales towards the satellite.
 - This was achieved using several different uniforms, with the vertexShader being used to scale the plane and the fragment shader being used to highlight the terrain. All uniforms are initially stored in the material, and are then called on by the custom shaders.
 - The position is achieved by taking the current world position of the satellite, extracting the x and y coordinates, and then finding the point on the mesh that lines up with those. This is updated every frame as to track where the satellite is, as the satellite is always moving

- **Delta Time)**

- Made a very simple clock and used `.getDelta()` to make the animation run off of delta time rather than being tied to framerate, so that it always animates at the same speed regardless of device.

- **Dillon Fiske**

- **Orthographic camera)**

- I made a separate camera that has a two-dimensional view of the scene. This camera is positioned above the planet and looks downwards at the planet to get a view of the satellite every time it orbits over the top of the planet.
- **Camera swapping function)**
 - I added an event listener to the window to pick up any keydown event, that is whenever the user presses a key on their keyboard. In this case, every time the user presses the key 't' on their keyboard it switches between the orthographic camera and the perspective camera.
- **Satellite model and rotation)**
 - To build the satellite, I first created an off-white cylinder to be used for the body of the satellite. Next, I created two grey rectangles to be used for the solar panels and set them to rotate continuously. Then, I created a satellite group constant and added the body and the solar panels to it to keep everything together while it orbits the planet. Finally, I used sine and cosine functions multiplied with the orbit radius to move the position of the satellite, thus creating an orbit for the satellite.
- **Imported a moon model - above and beyond)**
 - To import a model of the moon from NASA's website, I first created a DRACO loader and set its decoder path so that it was able to read the passed in .glb file. Next, I created a GLTF loader to use the DRACO loader and construct the moon in a moon data environment. Finally, I took the scene data from the moon data environment, positioned it, and scaled it before adding it to the current scene.